Course: IT202-008-S2025
Assignment: IT202 Milestone 1
Student: Carl E. (cle3)
Status: Submitted | Worksheet Progress: 88.29%
Potential Grade: 7.70/10.00 (77.00%)
Received Grade: 0.00/10.00 (0.00%)
Grading Link: https://learn.ethereallab.app/assignment/v3/IT202-008-S2025/it202-milestone-1/grading/cle3

Instructions

1. Refer to Milestone1 of this doc: https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cQ88EWVwfo/view
2. Ensure you read all instructions and objectives before starting.
3. Ensure you've gone through each lesson related to this Milestone
4. Switch to the Milestone1 branch
    1. `git checkout Milestone1` (ensure proper starting branch)
    2. `git pull origin Milestone1` (ensure history is up to date)
5. Fill out the below worksheet
    - Ensure there's a comment with your UCID, date, and brief summary of the snippet in each screenshot
    - Ensure proper styling is applied to each page
    - Ensure there are no visible technical errors; only user-friendly messages are allowed
6. Once finished, click "Submit and Export"
7. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`
    3. `git push origin Milestone1`
    4. On Github merge the pull request from `Milestone1` to `dev`
    5. On Github create a pull request from `dev` to `prod` and immediately merge. (This will trigger the `prod` deploy to make the heroku prod links work)
8. Upload the same PDF to Canvas
9. Sync Local
10. `git checkout dev`
11. `git pull origin dev`

Section #1: (2 pts.) Feature: User Will Be Able To Register A New Account

Task #1 (0.67 pts.) - Validation

Weight: *33.33%*
Objective: *Validation*

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Sub-Tasks:

Task #1 (0.33 pts.) - HTML Form/Validation

Combo Task:

Weight: *33.33%*
Objective: *HTML Form/Validation*

Details:

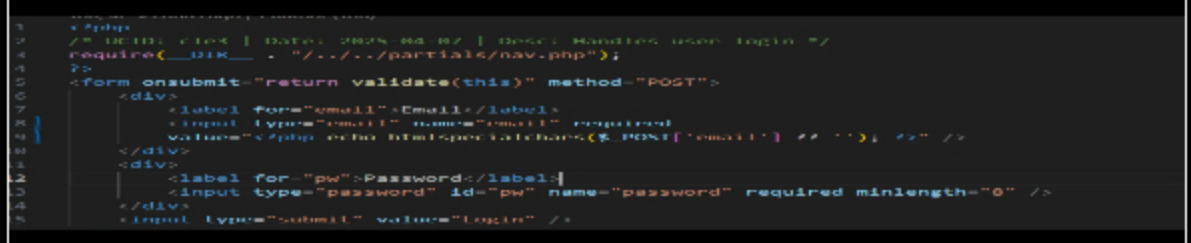
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Image Prompt

Weight: *50%*

Details:

- Show the code related to the register page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)



Showing the sticky validation

showing the form staying

Saved: 4/8/2025 4:49:40 PM

Text Prompt

Weight: *50%*

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I added sticky form fields to keep the email and username filled when the form fails. This uses PHP to save what you typed so you don't have to start over. The passwords don't stay because that's safer. browser checks first, then the server checks, and finally the database checks.

Saved: 4/8/2025 4:49:40 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Combo Task:

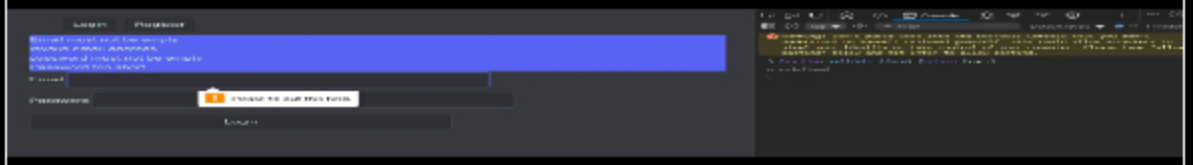
Weight: *33.33%*
Objective: *JS Validation (validate() function)*

Image Prompt

Weight: *50%*

Details:

- Show the code related to the register page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm](you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



validation, with the novalidate form

Saved: 4/8/2025 5:05:25 PM

Text Prompt

Weight: *50%*

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

I made the validate() function check for: valid email format, proper username (3-30 chars), 8+ character passwords, and matching confirm field. When something's wrong, it shows clear alerts like "Password needs 8+ chars". I added novalidate to the form to test this the screenshot shows those alerts working.

Saved: 4/8/2025 5:05:25 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call

Combo Task:

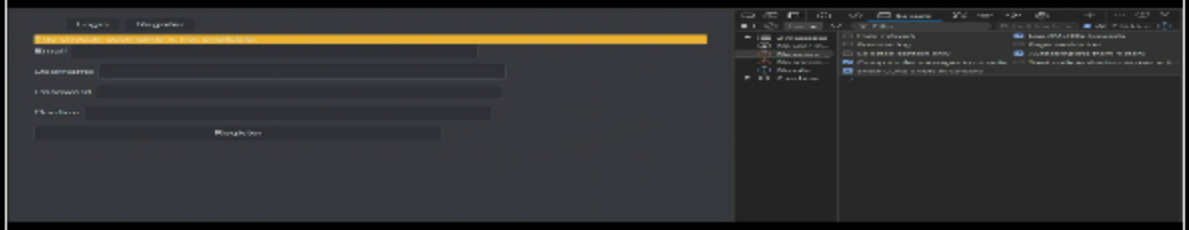
Weight: *33.33%*
Objective: *PHP Validation (steps before the DB call)*

Image Prompt

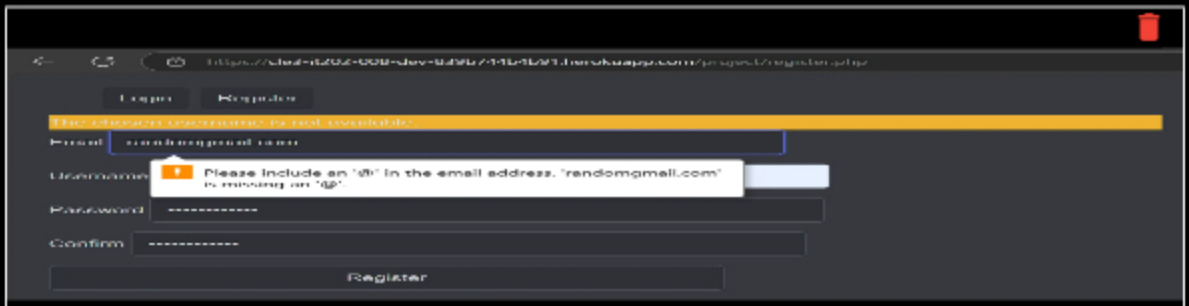
Weight: *50%*

Details:

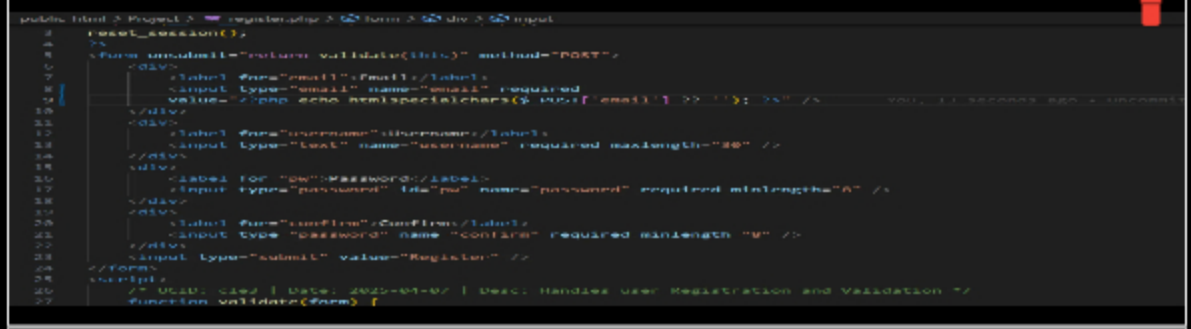
- Show the code related to the register page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to username/email already in use



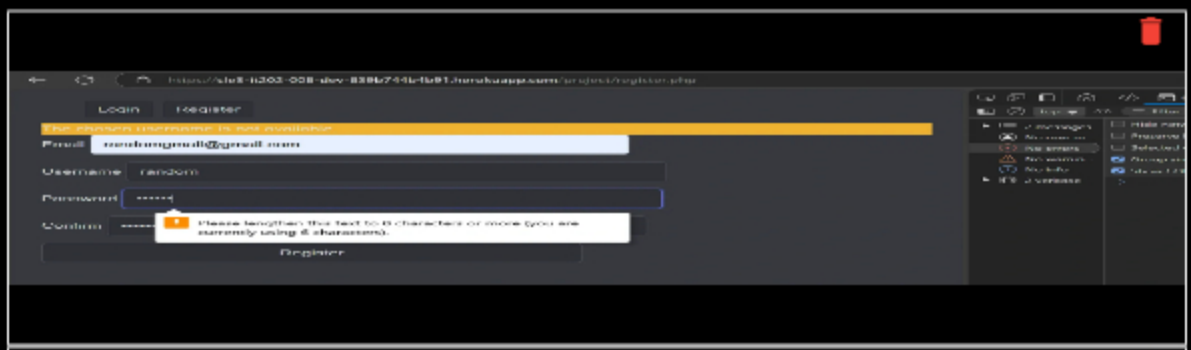
Showing example of invalid username



Showing example of invalid email



Showing register page's php validation



Showing example of invalid password message

Task #2 (0.67 pts.) - Related URLs

Url Prompt

Weight: *33.33%*
Objective: *Related URLs*

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://cle3-it202-008-dev-839b744b4b91.herokuapp.com/project/register.php

URL
https://cle3-it202-008-dev-839b744b4b

URL #2
https://cle3-it202-008-dev-839b744b4b91.herokuapp.com/project/login.php

URL
https://cle3-it202-008-dev-839b744b4b

Task #3 (0.67 pts.) - Database - Users table

Image Prompt

Weight: *33.33%*
Objective: *Database - Users table*

Details:

- Add a screenshot of the database view from vs code showing a valid registered user (preferrably a recent one during evidence capturing)

Showing valid user databse

Saved: 4/8/2025 8:43:17 PM

Section #2: (2 pts.) Feature: User Will Be Able To Login To Their Account

Task #1 (0.67 pts.) - Validation

Weight: *33.33%*
Objective: *Validation*

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Sub-Tasks:

Task #1 (0.33 pts.) - HTML Form/Validation

Combo Task:

Weight: *33.33%*
Objective: *HTML Form/Validation*

Details:

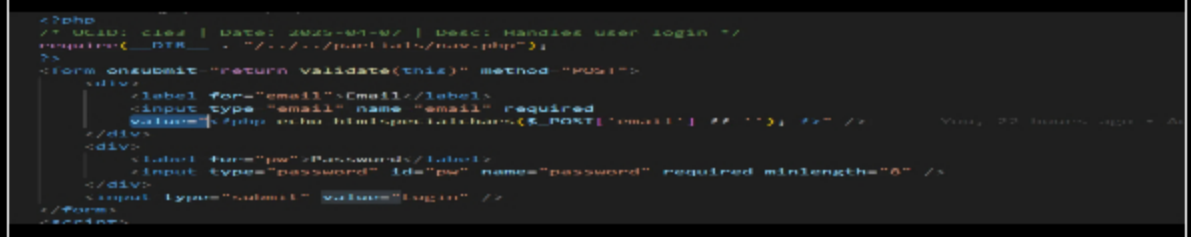
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Image Prompt

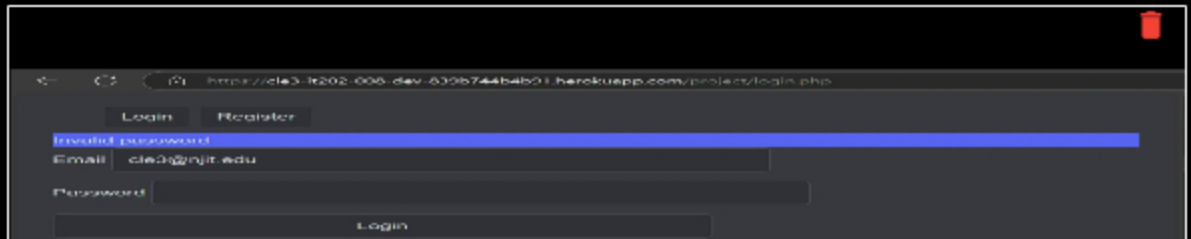
Weight: *50%*

Details:

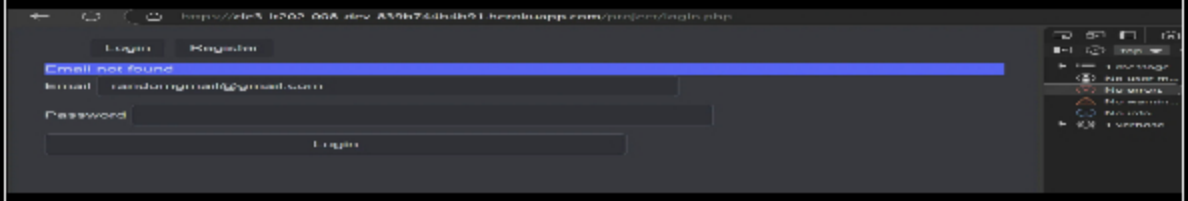
- Show the code related to the login page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)



Showing login page's form for php validation.



Sh



Showing examples of validation

Saved: 4/8/2025 8:57:00 PM

Text Prompt

Weight: *50%*

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Task #2 (0.33 pts.) - JS Validation (validate() function)

Combo Task:

Weight: *33.33%*
Objective: *JS Validation (validate() function)*

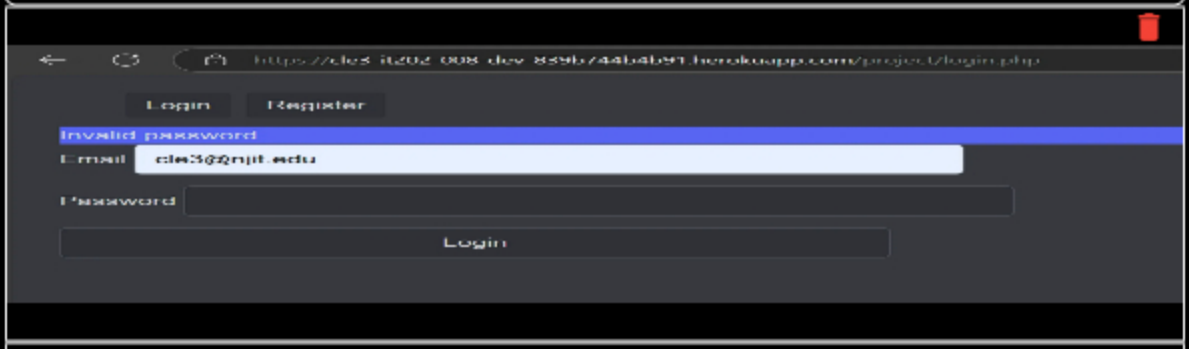
Image Prompt

Weight: *50%*

Details:

- Show the code related to the login page's validate() function
- Show examples of each validation message [username format, email format, password format] (you may be able to capture this in one or few screenshots)

- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag



showing login pages validate form

Showing example of login page's validate form

Saved: 4/8/2025 8:59:35 PM

Text Prompt

Weight: *50%*

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

Similar to the above, the validation checks for the correct parameters. Ensure the user has the correct email(checks for an @), and the password meets the min char length. If anything fails, a sticky form is deployed and a flash message is sent.

Saved: 4/8/2025 8:59:35 PM

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call

Combo Task:

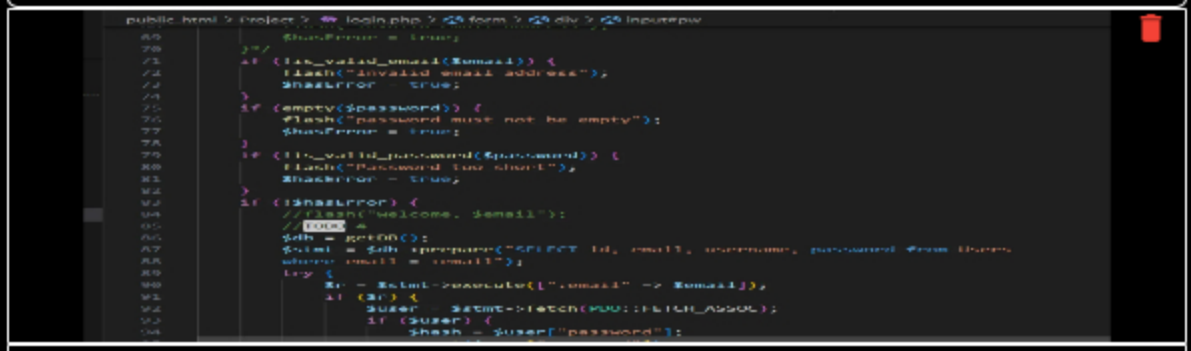
Weight: *33.33%*
Objective: *PHP Validation (steps before the DB call)*

Image Prompt

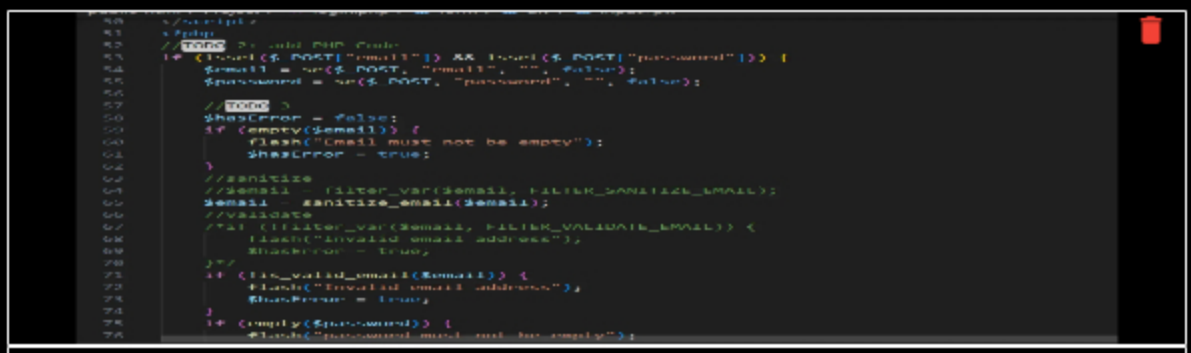
Weight: *50%*

Details:

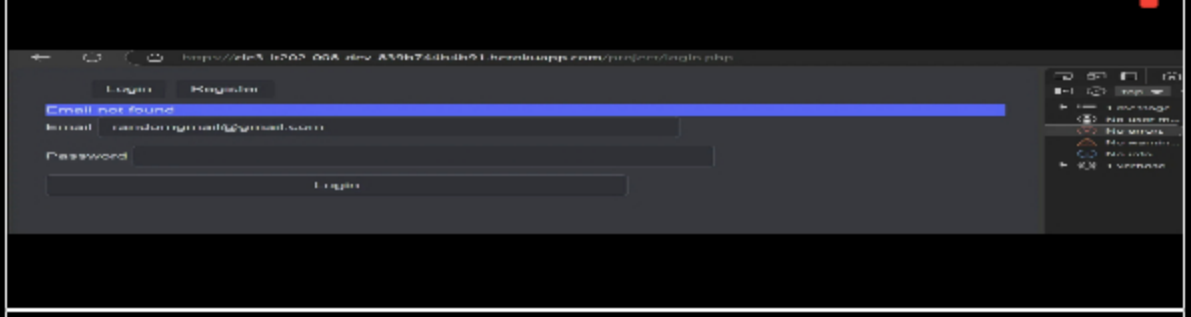
- Show the code related to the login page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions
- Include the outputs related to user doesn't exist and php-side password mismatch



Login page, php validation



Login page php validation



Invalid email

Saved: 4/8/2025 9:04:03 PM

Text Prompt

Weight: *50%*

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Task #2 (0.67 pts.) - Related URLs

Url Prompt

Weight: *33.33%*
Objective: *Related URLs*

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://cle3-it202-008-dev-839b744b4b91.herokuapp.com/project/login.php

URL
https://cle3-it202-008-dev-839b744b4t

URL #2
https://cle3-it202-008-prod-58562394d633.herokuapp.com/project/login.php

URL
https://cle3-it202-008-prod-58562394d

Task #3 (0.67 pts.) - User Session Evidence

Image Prompt

Weight: *33.33%*
Objective: *User Session Evidence*

Details:

```
- From the heroku logs (Dashboard -> More button -> view Logs), capture the session
error_log() output
- It should show the id, username, email, and roles
```

Heroku logs

Section #3: (1 pt.) Feature: User Will Be Able To Logout

Task #1 (1 pt.) - Evidence

Combo Task:

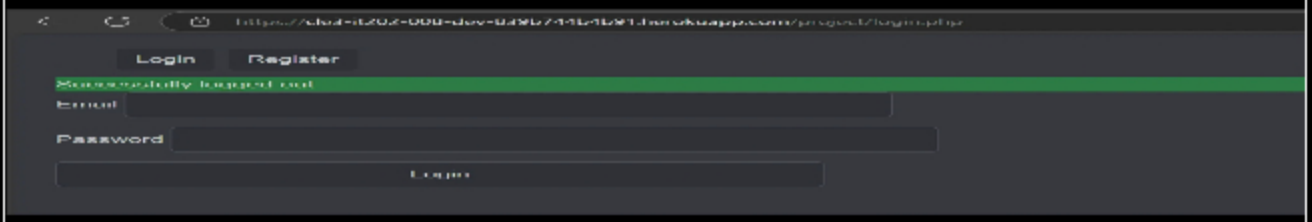
Weight: *100%*
Objective: *Evidence*

Image Prompt

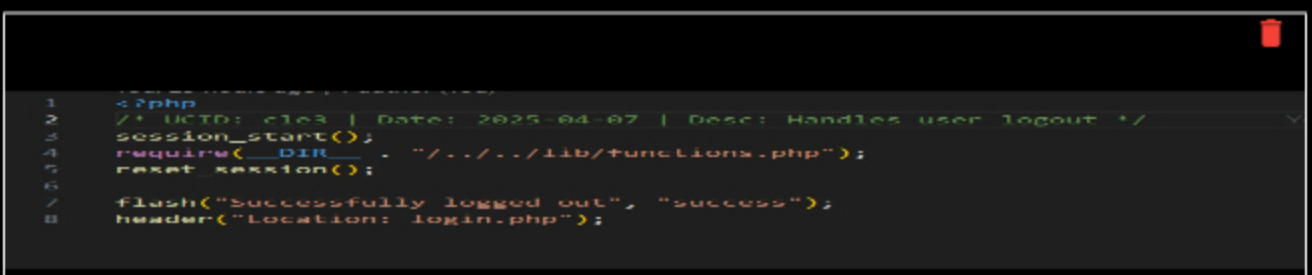
Weight: *40%*

Details:

- Show the successful logout message
- Show the code snippet of the logout page



showing successful logout message



```php
<?php
/* UCID: cle3 | Date: 2025-04-07 | Desc: Handles user logout */
session_start();
require(__DIR__ . "/../../lib/functions.php");
reset_session();

flash("Successfully logged out", "success");
header("Location: login.php");
```

VsCode logout page

Saved: 4/8/2025 9:35:18 PM

Url Prompt

Weight: *20%*

Details:

- Include the pull request url for this feature

URL #1

https://github.com/carledouard-dev/cle3-IT202-d020

https://github.com/carledouard-dev

Saved: 4/8/2025 9:35:18 PM

Text Prompt

Weight: *40%*

Details:

- Briefly explain how the logout process works and how the user is officially logged off

Your Response:

The logout process deletes the users session information, which ensures no security and sensitive data is at risk; and officially logs the user out by clearing their session credentials.

Saved: 4/8/2025 9:35:18 PM

Section #4: (2 pts.) Feature: Security Rules

Task #1 (1 pt.) - Database Tables

Image Prompt

Weight: *50%*
Objective: *Database Tables*

Details:

```
- From the vs code mysql tool, show both the Roles and UserRoles tables with data in them
```

showing UserRoles

Task #2 (1 pt.) - Demo Security Rules

Combo Task:

Weight: *50%*
Objective: *Demo Security Rules*

Image Prompt

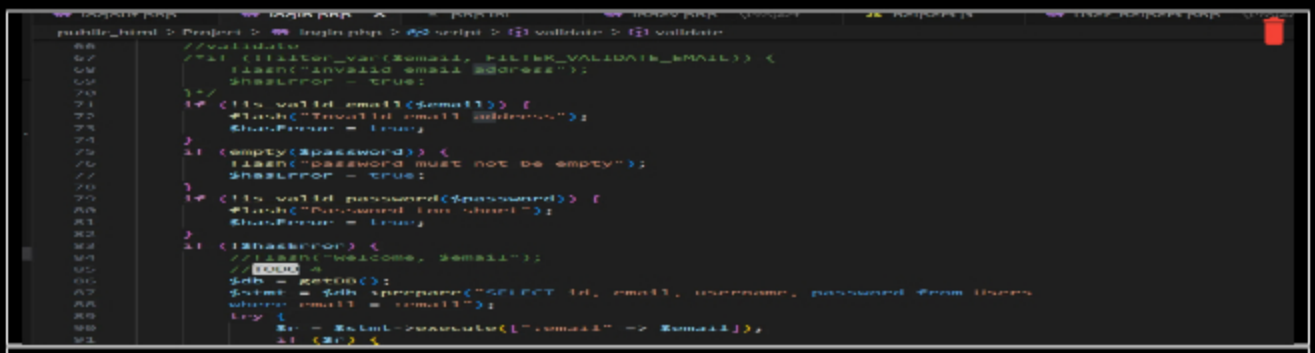
Weight: *40%*

Details:

```
  - Show the message related to needing to be logged in (i.e., try to manually
access a login protected page while logged out)
  - Show the message related to not having the proper permission/role (i.e., try
to manually access a role protected page while not having the proper role)
  - Include a snippet of the login check function
  - Include a snippet of the role check function
```

showing needing to be logged in

showing code snipped role check



showing login check

Saved: 4/8/2025 9:53:16 PM

Url Prompt

Weight: *20%*

Details:

- Include the pull request url for this feature

URL #1
https://github.com/carledouard-dev/cle3-
IT20

URL
https://github.com/carledouard-dev

Saved: 4/8/2025 9:53:16 PM

Text Prompt

Weight: *40%*

Details:

- Briefly explain how the login check code works
- Briefly explain how the role check code works

Your Response:

The way the login check works is that first, it validates the email format and password length, then

Section #5: (2 pts.) Feature: User Profile

Task #1 (1 pt.) - Validation

Weight: *50%*
Objective: *Validation*

Details:

- Show the relevant code snippets for each validation layer
- Show the relevant demo of each validation layer
- Briefly explain how the validation steps work at each layer

Sub-Tasks:

Task #1 (0.33 pts.) - HTML Form/Validation

Combo Task:

Weight: *33.33%*
Objective: *HTML Form/Validation*

Details:

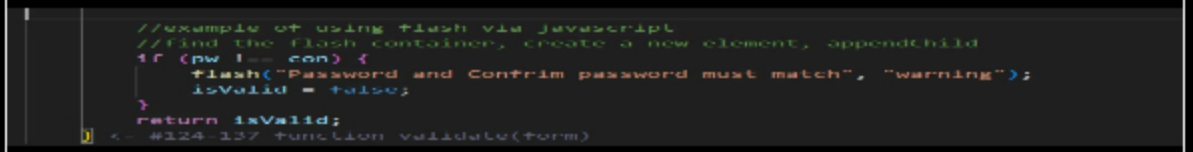
- System should allow the user to correct the error without wiping/clearing the form (for the PHP side this includes sticky-form logic to keep the username/email address entries)

Image Prompt

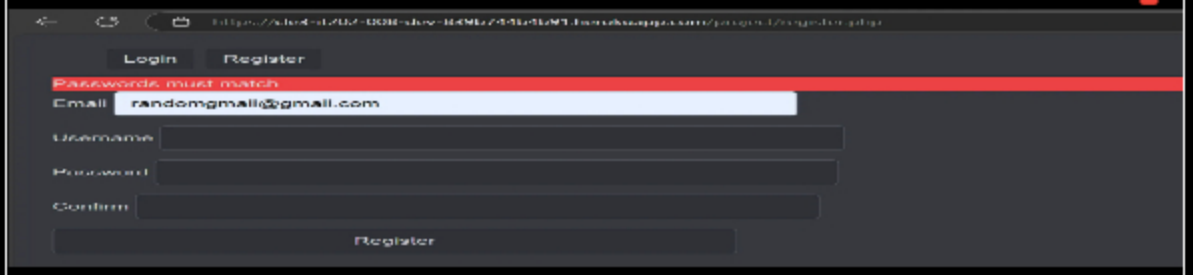
Weight: *50%*

Details:

- Show the code related to the profile page's form (HTML validation)
- Show examples of each validation message (you may be able to capture this in one or few screenshots)



code related to profile pages form validation(html)



Password validation

Saved: 4/8/2025 9:58:12 PM

Text Prompt

Weight: *50%*

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Your Response:

The code checks if the passwords match before submission to prevent unnecessary server requests.

Saved: 4/8/2025 9:58:12 PM

Task #2 (0.33 pts.) - JS Validation (validate() function)

Combo Task:

Weight: *33.33%*
Objective: *JS Validation (validate() function)*

Image Prompt

Weight: *50%*

Details:

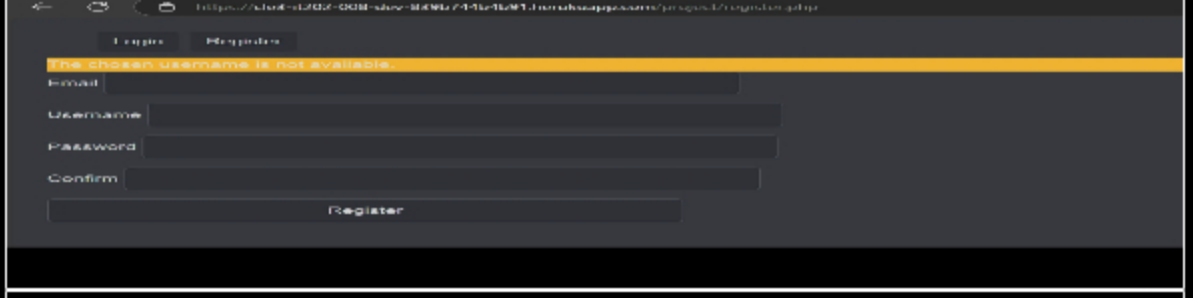
- Show the code related to the profile page's validate() function
- Show examples of each validation message [username format, email format, password format, password doesn't match confirm] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag

```
<script>
    function validate(form) {
        let pw = form.newPassword.value;
        let con = form.confirmPassword.value;
        let isValid = true;
        //TODO add other client side validation....

        //example of using flash via javascript
        //find the flash container, create a new element, appendChild
        if (pw !== con) {
            flash("Password and Confrim password must match", "warning");
            isValid = false;
        }
        return isValid;
    } <- #124-137 function validate(form)
</script>
<?php
require_once(__DIR__ . "/../../partials/flash.php");
?>
```

JS validation code

Passwords dont match



Username unavailable

Saved: 4/8/2025 10:00:53 PM

Text Prompt

Weight: *50%*

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Task #3 (0.33 pts.) - PHP Validation (steps before the DB call

Combo Task:

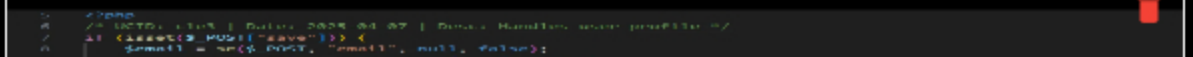
Weight: *33.33%*
Objective: *PHP Validation (steps before the DB call)*

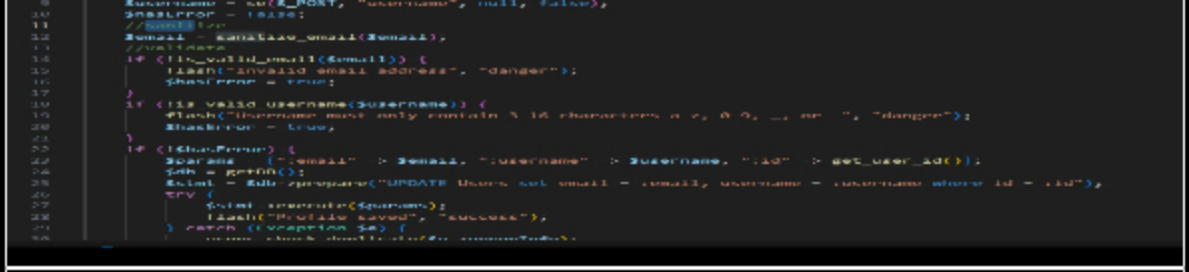
Image Prompt

Weight: *50%*

Details:

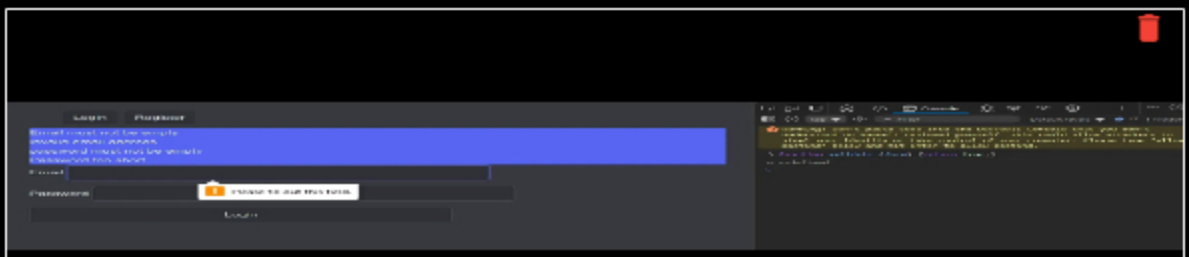
- Show the code related to the profile page's PHP validation
- Show examples of each validation message [username format, email format, password format, invalid password, password doesn't match confirm, username taken, email taken] (you may be able to capture this in one or few screenshots)
- Note: You may need to use dev tools to temporarily apply `novalidate` to the form tag and temporarily override the validate() function or use the provided helper script in the instructions





PHP validation

PHP validation



Showing no validate

Saved: 4/8/2025 10:04:56 PM

Text Prompt

Weight: *50%*

Details:

- Briefly explain the validation step (i.e, what you chose, how they solve the requirement)

Task #2 (1 pt.) - Related URLs

Url Prompt

Weight: *50%*
Objective: *Related URLs*

Details:

- Include the direct link to this file from the Milestone branch (should end in .php)
- Include the heroku prod link to this file (Just grab the base prod url and manually enter the path to the file)
- Include the related pull request link for this feature

URL #1
https://github.com/carledouard-dev/cle3-IT202-008/...

URL
https://github.com/carledouard-dev/cl

URL #2
https://cle3-it202-008-dev-839b744b4b91.herokuapp.com/project/register.php

URL
https://cle3-it202-008-dev-839b744b4b

Section #6: (1 pt.) Misc

Task #1 (0.33 pts.) - Github Details

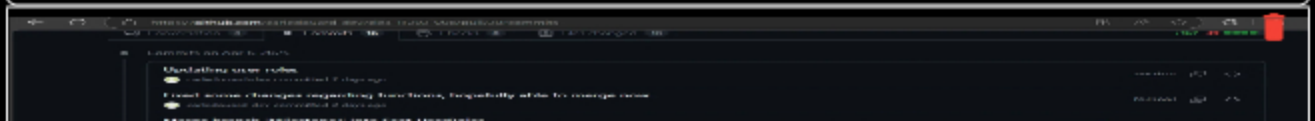
Combo Task:

Weight: *33.33%*
Objective: *Github Details*

Image Prompt

Weight: *60%*

Details:
From the Commits tab of the Pull Request screenshot the commit history



Commit history

Commit History

Saved: 4/8/2025 10:07:19 PM

Url Prompt

Weight: *40%*

Details:
Include the link to the Pull Request (should end in `/pull/#`)

Task #2 (0.33 pts.) - WakaTime - Activity

Image Prompt

Weight: *33.33%*
Objective: *WakaTime - Activity*

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time
- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary

WakaTime history

Task #3 (0.33 pts.) - Reflection

Weight: *33.33%*
Objective: *Reflection*

Sub-Tasks:

Task #1 (0.33 pts.) - What did you learn?

Text Prompt

Weight: *33.33%*
Objective: *What did you learn?*

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

Honestly, I learned a lot about databases, queries, SQL, and mainly PHP. I went into this having no prior experience using VsCode, and I am better than when I started. I even learned Regex, though I need some practice. The biggest thing was the authentication, I learned how to implement proper security measures and how to utilize different forms of authentication when submitting my page

Saved: 4/8/2025 10:10:20 PM

Task #2 (0.33 pts.) - What was the easiest part of the assignr

Text Prompt

Weight: *33.33%*
Objective: *What was the easiest part of the assignment?*

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The easiest part of the assignment would have to be the beginning. When everyone registered for Heroku, and set up VsCode. That was by far the simplest lol.

Saved: 4/8/2025 10:10:48 PM

Task #3 (0.33 pts.) - What was the hardest part of the assign

Text Prompt

Weight: *33.33%*
Objective: *What was the hardest part of the assignment?*

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The hardest part of the assignment was the novalidate, learning how to implement the right JS and HTML validations, and using dev tools to paste them. It wasn't too bad but it made me think. Also the admin links for my navigation path that was something as well!

Saved: 4/8/2025 10:11:46 PM